CSE2115 Peer Assignment 5 - software maintenance

REDACTED

Technische Universiteit Delft

1. Cyclomatic Complexity

1.1. A

CC = 12; Count every if/else if (not just else) plus every && |, then add 1:

- 8 ’if’/’else if’ conditions
  - ´STANDARD’
  - ´US’
  - ´total>100’
  - ´else if EU’
  - ´total>50 && PREMIUM’
  - ´else if EXPRESS’
  - ´US||CA’
  - ´cost>0 && HEAVY’
- 3 logical operators
  - the ’&&’ in the EU check
  - the ’||’ in the US/CA check
  - the ’&&’ in the surcharge check

1.2. B

- *Hard to test* - CC is roughly the minimum number of test cases needed for full branch coverage, so here you’d need ˜12 tests just to hit every path once. In practice people don’t write all of them, so branches go untested and bugs slip through.
- *Hard to change safely / easy to hide bugs* - this code already proves it: the EXPRESS branch checks ‘shippinngSpeed‘ (typo mistake, extra "n"), not ‘shippingSpeed‘. In a method this deeply nested, a one-letter typo like that is easy to miss during review, and the EXPRESS branch ends up broken without anyone noticing.

1.3. C

- **Extract Method** - split out ‘calculateStandardCost(order, user)‘, ‘calculateExpressCost(order, user)‘, and ‘applyHeavyItemSurcharge(cost, order)‘, so ‘calculateShippingCost‘ just manages them.
- **Replace Conditional with Polymorphism** - define a ‘ShippingStrategy‘ interface with ‘calculate(order, user)‘, implement ‘StandardShippingStrategy‘ and ‘ExpressShippingStrategy‘, and pick the right one from a ‘Map<String, ShippingStrategy>‘ keyed by ‘shippingSpeed‘. Adding a new shipping speed later means adding a new class, not touching this method at all.

2. Lack of cohesion method

2.1. A

Attribute usage per method:

- ‘readFile‘: filename, fileContent
- ‘parseContent‘: fileContent, recordCount
- ‘connectToDatabase‘: databaseUrl
- ‘saveToDatabase‘: databaseUrl, recordCount

Pair	Shared attribute?
readFile & parseContent	yes (fileContent)
readFile & connectToDatabase	no
readFile & saveToDatabase	no
parseContent & connectToDatabase	no
parseContent & saveToDatabase	yes (recordCount)
connectToDatabase & saveToDatabase	yes (databaseUrl)

- 3 P (non-sharing pairs)
- 3 Q (sharing pairs)
- LCOM = max(0, P - Q) = 0

2.2. B

LCOM equal to 0 suggests the class is fully cohesive, but that is not correct. The class actually does two separate things: reading/parsing a file (readFile, parseContent) and saving to a database (connectToDatabase, saveToDatabase). They only look connected because recordCount and databaseUrl happen to be shared. One shared field is enough to hide this in the LCOM score, so it still looks like a Large Class. To fix this, we should split it into two classes: FileParser(readFile, parseContent) and DatabaseWriter(connectToDatabase, saveToDatabase). The parser returns the record count and the writer takes it as a parameter. Each class has one job and is easier to test. The downside is that you now have two objects to connect instead of one.

3. Technical debt

3.1. A

- **Magic numbers with no default case** - ‘format == 1‘ / ‘format == 2‘ with nothing handling any other value (the method silently builds ‘report‘ and throws it away). Future cost: anyone extending this has to go hunting for what 1 and 2 mean, and passing a bad value fails silently instead of erroring.
- **Mixed responsibilities in one method** - building the report text and converting/saving it per format are two different jobs stuffed into one method. Future cost: the method only grows as formats are added, gets harder to test (you’d need to mock file I/O just to check the text formatting), and touching it for a new format risks breaking the existing ones.

3.2. B

Introduce a ‘ReportFormatter‘ interface, with ‘HtmlReportFormatter‘ and ‘PdfReportFormatter‘ implementations:

CODE BELOW IS AI GENERATED FOR SAKE OF ILLUSTRATION

```java
public interface ReportFormatter {
    void export(String report);
}

public class HtmlReportFormatter implements ReportFormatter {
    public void export(String report) { /* convert to HTML, save */ }
}

public class PdfReportFormatter implements ReportFormatter {
    public void export(String report) { /* convert to PDF, save */ }
}
```

‘generateReport‘ takes a ‘ReportFormatter‘ instead of an ‘int‘:

CODE IS GENERATED

```java
public void generateReport(String title, String data, ReportFormatter formatter) {
    String report = buildReportContent(title, data); // header/body/footer logic
    formatter.export(report);
}
```

Anytime you want to add a new format, simply write a new class implementing ReportFormatter. This way nothing in generateReport changes.

4. Open-Source Analysis: Queue

4.1. A

We chose Queue (TUDelft lab queue app), specifically the LabController.java class. [1] The class has many methods that are almost copies of each other. For example:

```java
public String createRegularLab(Long editionId, RegularLabCreateDTO dto) {
    ls.createSessions(dto, editionId, REGULAR);
    return "redirect:/edition/" + editionId + "/labs";
}
public String createSlottedLab(Long editionId, SlottedLabCreateDTO dto) {
    ls.createSessions(dto, editionId, SLOTTED);
    return "redirect:/edition/" + editionId + "/labs";
}
```

The same pattern appears for exam and capacity labs, and for the shared-edition versions (7 create methods in total). The ’updateSession’ method for regular, slotted and exam labs also have exactly the same body.

4.2. B (Code issues)

- **Duplicate code:** multiple create methods and multiple update methods only differ in the DTO type.
- **Large Class:** the file is around a 1000 lines with 20 ish ‘@Autowired‘ dependencies, and it handles creating, updating, deleting, importing, exporting and enqueueing.

[1] https://gitlab.ewi.tudelft.nl/eip/labrador/queue/-/blob/development/src/main/java/nl/tudelft/queue/controller/LabController.java

- **Impropper utilization of polymorphism:** every new lab type needs another copy of the same methods.

4.3. C

The main technique is Extract Method. Put the shared creation code in one helper, and let each create end-point just call it:

CODE IS GENERATED
“‘java
private String createLab(SessionCreateDTO dto, Long id, boolean shared) { ... }
“‘

The 3 identical updateSession methods can be merged the same way.

4.4. D

A change to how labs are created or updated (new permission check, different redirect) then only has to be made once, instead of in 7 or more places that could get out of sync. The controller also gets shorter and easier to read. It doesn’t fix the Large Class problem fully, because that would need splitting the controller up.